МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №5

з курсу: „Обробка надвеликих масивів даних”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Метою досліджень по цій темі є отримання практичних навичок по токензації текстів як етапу до створення вбудовувань (embeddings). Вбудовування лежать в основі навчання великих мовних моделей (LLM). В цій темі пропонується вивчити і дослідити техніки реалізації двох етапів токенізації: побудови токенів текстів і визначення ідентифікаційних номерів (ID) токенів, а також створення словників та токенізаторів в системах обробки LLM.

Завдання.

1. Познайомитися з методами побудови словників і токенізаторів, які описані в даній інструкції, детально проаналізувати наведені приклади.
2. Вибрати самостійно кілька простих прикладів текстів для виконання експериментів.
3. Створити словник і токенізатор, які здатні обробляти вибрані тексти.
4. Виконати комп’ютерні експерименти з вибраними текстами, описані в даній інструкції, з використанням створених словника і токенізатора.
5. Представити повний письмовий звіт з виконаних досліджень

Хід роботи

1. Познайомитися з методами побудови словників і токенізаторів, які описані в даній інструкції, детально проаналізувати наведені приклади.

Після того як ми розібралися з основними підходами до створення словників і токенізаторів, можна переходити до практичної частини - самостійного виконання наведених у лабораторній роботі інструкцій.

Копіюємо текст оповідання «Вердикт» у файл “the-verdict.txt”:

```
import urllib.request
url = ("https://raw.githubusercontent.com/rasbt/"
"LLMs-from-scratch/main/ch02/01_main-chapter-code/"
"the-verdict.txt")
file_path = "the-verdict.txt"
urllib.request.urlretrieve(url, file_path)
```

В результаті отримуємо файл the-verdict.txt:

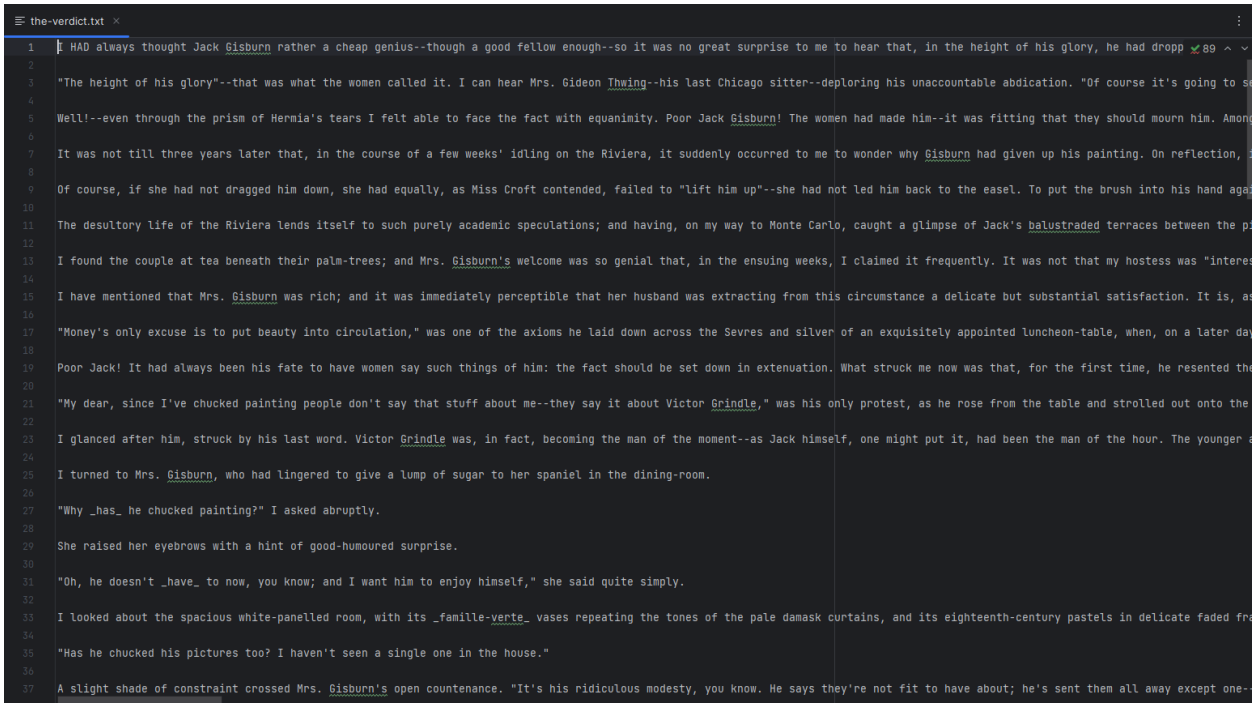


Рис.1 Оповідання «Вердикт» у файл the-verdict.txt

Відкриваємо файл для читання, читаємо перші 100 знаків і висвітлюємо їх на екрані монітора:

```
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()
print("Total number of character:", len(raw_text))
print(raw_text[:99])
```

```
Total number of character: 20479
I HAD always thought Jack Gisburn rather a cheap genius--though a good fellow enough--so it was no
```

Рис.2 Повна кількість символів та перші 100 символів

Виконаємо токенізацію тексту:

```
import re

with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()
preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
print("Кількість токенів: ", len(preprocessed))
print("Перші 30 токенів: ", preprocessed[:30])
```

```
Кількість токенів:  4690
Перші 30 токенів:  ['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'genius', '--', 'though', 'a', 'good', 'fellow', 'enough', '--', 'so',
```

Рис.3 Результат роботи токенайзера

Побудуємо словник токенів:

```
all_words = sorted(set(preprocessed))
vocab_size = len(all_words)
print(vocab_size)

vocab = {token:integer for integer,token in enumerate(all_words)}
for i, item in enumerate(vocab.items()):
    print(item)
    if i >= 50:
        break
```

В результаті отримуємо словник, що містить токени і відповідні до них числові мітки - цілі числа, тобто ID токенів.

```
Перші 30 токенів:  ['I', 'HAD', 'always', 'thought', 'Jack', 'Gisburn', 'rather', 'a', 'cheap', 'genius', '--', 'though',
1130
('!', 0)
('"', 1)
("'", 2)
('(', 3)
(')', 4)
(',', 5)
('--', 6)
('.', 7)
(':', 8)
(';', 9)
('?', 10)
('A', 11)
('Ah', 12)
('Among', 13)
('And', 14)
('Are', 15)
('Arrt', 16)
('As', 17)
('At', 18)
('Be', 19)
('Begin', 20)
('Burlington', 21)
('But', 22)
('By', 23)
('Carlo', 24)
('Chicago', 25)
('Claude', 26)
('Come', 27)
('Croft', 28)
('Destroyed', 29)
('Devonshire', 30)
('Don', 31)
```

Рис.4 Словник токенів

Створимо простий клас токенізатора:

```
class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i:s for s,i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        preprocessed = [
```

```
        item.strip() for item in preprocessed if item.strip()
        ]
        ids = [self.str_to_int[s] for s in preprocessed]
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str[i] for i in ids])
        text = re.sub(r'\s+([,.?!"()\'])', r'\1', text)
        return text
```

Протестуємо його роботу на прикладі тексту з файлу the-verdict.txt:

```
Кодування:  [21, 1005, 921, 796, 745, 97, 531, 506, 628, 1066, 947, 903, 506, 544, 828, 769, 947, 278, 957, 50, 1137,
Декодування:  I HAD always thought Jack Gisburn rather a cheap genius -- though a good fellow enough -- so it was no
```

Рис.5 Кодування та декодування тексту токенізатором

При спробі застосування його до нового зразка тексту, який не міститься в навчальному наборі

```
text = "Hello, do you like tea?"
print(tokenizer.encode(text))
```

Отримаємо повідомлення про помилку через відсутність цього слова в оповіданні вердикт

```
KeyError: 'Hello'
```

Рис.6 Помилка кодування

2. Створити словник і токенізатор, які здатні обробляти вибрані тексти.

Створимо словник і токенізатор за прикладом з інструкцій, додамо в токенізатор можливість обробки слів, що відсутні в словнику:

```
preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', raw_text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
all_words = sorted(set(preprocessed))
vocab = {token:integer for integer,token in enumerate(all_words)}

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.unk_token = "<UNK>"

        if self.unk_token not in vocab:
            vocab[self.unk_token] = len(vocab)

        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        preprocessed = [item.strip() for item in preprocessed if
item.strip()]
```

```
        ids = []
        for token in preprocessed:
            if token in self.str_to_int:
                ids.append(self.str_to_int[token])
            else:
                ids.append(self.str_to_int[self.unk_token])
        return ids

    def decode(self, ids):
        text = " ".join([self.int_to_str.get(i, self.unk_token) for i in
ids])
        text = re.sub(r'\s+([,.?!"()\'])', r'\1', text)
        return text
```

Протестуємо його на обраному тексті:

```
tokenizer = SimpleTokenizerV1(vocab)

encode = tokenizer.encode(raw_text)
print("Кодування: ", encode)
print("Декодування: ", tokenizer.decode(encode))

text = "Hello, lorem dolore augue?"

encode = tokenizer.encode(text)
print("Кодування: ", encode)
print("Декодування: ", tokenizer.decode(encode))
```

```
Кодування:  [10, 95, 154, 121, 103, 114, 133, 121, 81, 69, 65, 74, 147, 72, 116, 0, 58, 81, 134, 0, 52, 113, 131, 0,
Декодування:  Dolor est tempor lorem facilisis iusto nonummy lorem dolore augue aliquyam consectetuer sed clita kasd.
Кодування:  [167, 167, 121, 81, 69, 167]
Декодування:  <UNK> <UNK> lorem dolore augue <UNK>
```

Рис.7 Кодування та декодування тексту токенізатором

3. Виконати комп’ютерні експерименти з вибраними текстами, описані в даній інструкції, з використанням створених словника і токенізатора.

Створимо декілька тестів та застосуємо їх до тексту:

```
print("\nТЕСТ 1: Пробіли vs Без пробілів\n")
sample_words = preprocessed[:10]
sentence_with_spaces = " ".join(sample_words)
sentence_no_spaces = "".join(sample_words)

tokens_with_spaces = tokenizer.encode(sentence_with_spaces)
tokens_no_spaces = tokenizer.encode(sentence_no_spaces)

print("Текст з пробілами:\n", sentence_with_spaces)
print("К-сть токенів:", len(tokens_with_spaces))
print("Токени:", tokens_with_spaces)

print("\nТекст без пробілів:\n", sentence_no_spaces)
print("К-сть токенів:", len(tokens_no_spaces))
print("Токени:", tokens_no_spaces)


print("\nТЕСТ 2: Регістр vs lower()\n")
```

```
sentence_case = " ".join(preprocessed[:12])

tokens_case = tokenizer.encode(sentence_case)

pre_lower = [t.lower() for t in preprocessed]
vocab_lower = {tok: i for i, tok in enumerate(sorted(set(pre_lower)))}
tokenizer_lower = SimpleTokenizerV1(vocab_lower)

tokens_case_lower = tokenizer_lower.encode(sentence_case.lower())

print("Оригінальна фраза:")
print(sentence_case)
print("\nОригінальні токени:", tokens_case)

print("\nФраза в lower:")
print(sentence_case.lower())
print("Lower токени:", tokens_case_lower)

print("\nРозмір словника з регістром:", len(vocab))
print("Розмір словника без регістру (lower):", len(vocab_lower))

print("\nТЕСТ 3: Пунктуація vs Без пунктуації\n")
sentences = re.split(r'[.!?]', raw_text)
sentence_raw = sentences[0].strip()

sentence_no_punct = re.sub(r'[,.:;!?()"\'-]', '', sentence_raw)

tokens_punct = tokenizer.encode(sentence_raw)
tokens_no_punct = tokenizer.encode(sentence_no_punct)

print("Фраза з пунктуацією:")
print(sentence_raw)
print("Токени:", tokens_punct)

print("\nФраза без пунктуації:")
print(sentence_no_punct)
print("Токени:", tokens_no_punct)
```

У результаті проведених експериментів над текстом із файлу було досліджено вплив різних факторів - пробілів, регістру та пунктуації на роботу токенізатора. Результати підтвердили ключові властивості сегментації тексту та побудови словника.

```
ТЕСТ 1: Пробіли vs Без пробілів

Текст з пробілами:
 Dolor , est tempor lorem facilisis iusto nonummy lorem dolore
К-сть токенів: 10
Токени: [11, 0, 96, 155, 122, 104, 115, 134, 122, 82]

Текст без пробілів:
 Dolor,esttemporloremfacilisisiustononummyloremdolore
К-сть токенів: 3
Токени: [11, 0, 168]

ТЕСТ 2: Регістр vs lower()

Оригінальна фраза:
Dolor , est tempor lorem facilisis iusto nonummy lorem dolore augue aliquyam

Оригінальні токени: [11, 0, 96, 155, 122, 104, 115, 134, 122, 82, 70, 66]

Фраза в lower:
dolor , est tempor lorem facilisis iusto nonummy lorem dolore augue aliquyam
Lower токени: [21, 0, 36, 97, 62, 44, 55, 74, 62, 22, 10, 6]

Розмір словника з регістром: 169
Розмір словника без регістру (lower): 111

ТЕСТ 3: Пунктуація vs Без пунктуації

Фраза з пунктуацією:
Dolor, est tempor lorem facilisis iusto nonummy lorem dolore augue aliquyam consectetuer sed clita kasd
Токени: [11, 0, 96, 155, 122, 104, 115, 134, 122, 82, 70, 66, 75, 148, 73, 117]

Фраза без пунктуації:
Dolor est tempor lorem facilisis iusto nonummy lorem dolore augue aliquyam consectetuer sed clita kasd
Токени: [11, 96, 155, 122, 104, 115, 134, 122, 82, 70, 66, 75, 148, 73, 117]
```

Рис.8 Кодування та декодування тексту токенізатором

1. Вплив пробілів на кількість токенів тексти з пробілами розбиваються на коректну кількість слів і токенів, тоді як відсутність пробілів призводить до злиття слів у довгі токени. Пробіли є критично важливими для правильної сегментації та структури тексту.

2. Врахування регістру збільшує розмір словника, оскільки різні форми слова (наприклад, з великої літери та з малої) стають окремими токенами. Перехід у нижній регістр зменшує словник, але при цьому втрачається частина семантичної інформації, наприклад власні назви або початок речень.

3. Наявність пунктуації збільшує словник, оскільки знаки окремо враховуються як токени. Пунктуація допомагає зберігати граматику та структуру речень, підвищує точність декодування. Відсутність пунктуації зменшує словник, але текст стає менш структурованим і важчим для читання.

Робота токенізатора сильно залежить від способу попередньої обробки тексту. Для коректної роботи важливо враховувати пробіли, регістр та пунктуацію. Це дозволяє зберегти точність сегментації, адекватний розмір словника та семантичну структуру тексту.

Висновок:

У рамках лабораторної роботи було досліджено процес токенізації тексту, який є ключовим етапом підготовки даних для обробки природної мови. Було завантажено текст The Verdict, виконано його попередню обробку та розділено на окремі смислові одиниці. У ході роботи створено простий токенізатор, здатний перетворювати текст у числові послідовності та відновлювати текст із цих послідовностей.

Під час експериментів було перевірено різні підходи до токенізації, зокрема з урахуванням пробілів, пунктуації та регістру символів. Результати показали, що спосіб розділення тексту значно впливає на розмір словника та точність відображення змісту. Крім того, було розглянуто проблему невідомих токенів і реалізовано їх обробку через спеціальний токен <UNK>, що дозволяє уникати помилок при зустрічі нових слів.

Отримані дані підтвердили, що токенізація є важливою складовою підготовки текстових даних для подальшого аналізу та навчання моделей машинного навчання. Якість токенізації безпосередньо впливає на здатність моделі правильно сприймати структуру тексту та передавати його зміст у числовій формі. В результаті роботи було глибше засвоєно принципи представлення тексту у вигляді токенів і зрозуміло значення токенізатора у сучасних методах обробки природної мови.